Project Proposal — Voice-Command Keyboard

Name: Sahil Makim
Class: CS 3651 (Prototyping Intelligent Devices)

Description

I'm building a 40% mechanical keyboard, 12 by 4 matrix with 47 keys. This keyboard should work as a normal wired keyboard, but it adds a new voice-command mode. When a user presses a dedicated button, the keyboard listens to a I2S MEMs microphone to process voice commands. the voice command mode is an on-device keyword spotting model that classifies spoken phrases against a fixed list of commands. The recognized command is then sent to the computer as a keyboard shortcut; mimicing pressing the required keys in the required order for some specified output. As an example:

- "close tab" -> cmd+w
- "open brave" -> cmd + space, delay, 'brave' + enter
- "school mode" -> cmd + space, delay, brave new tab , delay, gatech.instructure.com + enter.
  The goal is to setup a voice command method to trigger software defined macros saved and setup in the microcontroller.

Speech recognition should run on the microcontroller itself, not on the host computer. This makes the device implement more processing than just sending all the information to the computer for processing.
So far the scope is restricted to just keyword spotting, as full text to speech cannot run on an MCU (maybe a command to start text to speech to run on the computer using the on board mems microphone).

Link List

Keyboard / matrix:

- Keyboard matrix circuit explanation — https://en.wikipedia.org/wiki/Keyboard_matrix_circuit
- Hand-wired keyboard build guide — https://www.instructables.com/Building-a-Handwired-Keyboard/
- QMK docs (reference for matrix/keymap concepts) — https://qmk.fm/

Firmware / USB:

- ESP-IDF (the SDK I'll write the firmware in) — https://docs.espressif.com/projects/esp-idf/en/latest/
- TinyUSB HID keyboard example (USB output) — https://github.com/hathach/tinyusb
- ESP32 keyboard firmware references (reference only, not porting): MK32, paul356/esp32_keyboard

On-device speech / keyword spotting (highest-risk element — two approaches):

- Option A: Espressif ESP-SR / MultiNet (prebuilt command recognizer for ESP32-S3) — https://github.com/espressif/esp-sr
- Option B: Edge Impulse keyword spotting — https://docs.edgeimpulse.com/docs/
- Background reference: Google Speech Commands dataset — https://www.tensorflow.org/datasets/catalog/speech_commands

Hardware:

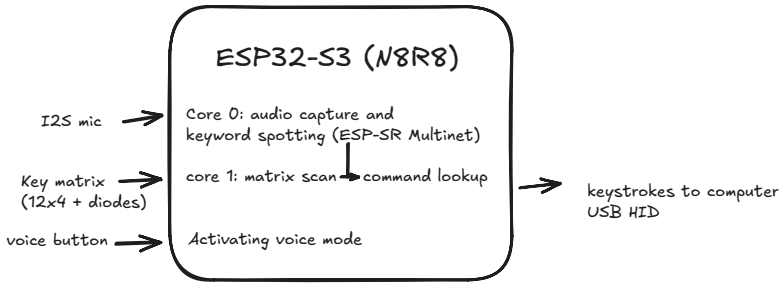
- Adafruit I2S MEMS microphone (SPH0645) — https://www.adafruit.com/product/3421
- Layout editor (KLE) — http://www.keyboard-layout-editor.com/
- VIA (keyboard remapping reference) — https://www.usevia.app/

Part List

Part	Purpose	Link	Est. qty
ESP32-S3-DevKitC-1-N8R8 (8MB flash / 8MB PSRAM)	Main MCU. PSRAM required by ESP-SR	Espressif · Mouser · DigiKey	1
I2S MEMS mic (INMP441 or Adafruit SPH0645)	Voice input	INMP441 (pre-soldered) · Adafruit SPH0645	1–2 (Professor mentioned lab supply)
MX-style mechanical switches (Alpaca V2)	Key switches	Some random Amazon switches	~47 (+ spares)
Amoeba King single-key PCBs	Hand-wire-friendly hot-swap (holds Kailh socket + diode)	keeb amoeba things	~47 (buy 2×25)
Kailh MX hot-swap sockets	Mount in each Amoeba King for hot-swap	splitkb · YMDK 50-pk, Amazon	~47
1N4148 diodes	Matrix anti-ghosting (if not on hot-swap boards)	DigiKey (onsemi) · bulk, Amazon	~60

Part	Purpose	Link	Est. qty
Push button	Voice-mode trigger	The nicer ones from the kit	1
24 AWG solid-core + enameled wire	Matrix hand-wiring	lab room	—
USB-C cable	Data + power	data-capable USB-C, I have already	1
2u plate-mount stabilizer	Spacebar	Durock V3	1???
Keycaps (40% set or blanks)	Caps	YMDK blank PBT	1 set
PLA/PETG filament, M2/M3 screws, heat-set inserts	3D-printed plate + case	in lab room	—

Block Diagram



- **Inputs:** key matrix (12×4) and I2S microphone; voice-mode push button.
- **Processing (on MCU):** matrix scan + debounce; on-device keyword spotting; command→shortcut lookup.
- **Outputs:** USB HID keystrokes to the computer.

Goals + Stretch

Core:

- Working 40% keyboard: full 12×4 matrix, debounce, function layer, USB HID, types normally on macOS.
- Voice mode: 10–15 distinct commands recognized **on-device** and sent as the correct shortcuts.
- 3D-printed plate + case housing all electronics.

Stretch:

- Larger command vocabulary; a custom wake word instead of a button.

- Layers / macros beyond the basic function layer.
- Custom PCB instead of hand-wiring.
- ability to use the keyboard with ANY device without requiring auxilary programs on computer would be nice, not sure on the feasibility.

Mid-project Review Deliverable

Hard deliverable: on a breadboard, the ESP32-S3 recognizes at least 5 spoken commands on-device and prints them, and a small test matrix (a few switches with diodes) sends correct keystrokes to the computer over USB HID.

This proves that the on-device speech recognition and USB HID output work on the real hardware, early enough to re-scope if keyword spotting struggles.

Potential changes

- **Speech approach fallback:** start with ESP-SR/MultiNet (Option A). If its fixed command set or accuracy is too limiting, fall back to a custom Edge Impulse / TFLite-Micro keyword-spotting model (Option B).
- **MCU:** the project *requires* an ESP32-S3 with PSRAM.
- **Scope reduction if needed:** cut command count, or switch from hold-to-talk to a simpler always-listening-while-pressed model.
- **Noise handling:** if background noise hurts accuracy, reduce the vocabulary to more acoustically distinct phrases and tune mic gain/placement.
- MAY SWITCH MID-PROJECT REVIEW TO HAVING AT LEAST THE KEYS WIRED, might be worth to just get the keys working first, at the least for sure get all the parts (will talk with professor on this).